

Control of Dynamic Systems Using Neural Networks

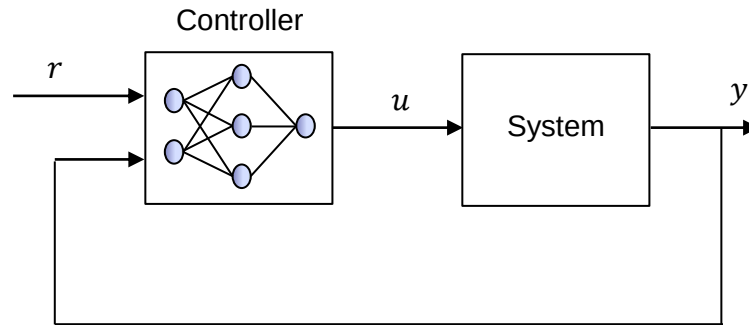
The objective of controlling a system is to make it behaves in a desired way, fulfilling a desired task, with a proper use of available resources. For that, the controller requires to know the system present and desired states to compare them for determining the appropriate control actions to be applied to the system for making it advances to the desired state, fulfilling restrictions and limitations, and within appropriate stability margins.

The objectives for controlling a dynamic system may be grouped in three categories:

- Stabilization control. An unstable dynamic system is controlled for making it BIBO stable: bounded input – bounded output. Starting from an initial state, an asymptotically stable system converges to an equilibrium state.
- Regulation control. A system is controlled for ensuring it operates in a set point (or set state) even in the presence of internal or external disturbances. Vibration isolation is an example of regulation control.
- Tracking control. A system is controlled for following (tracking) a desired or reference trajectory.

For unstable systems, controllers are firstly designed for solving the stabilization control problem, and afterwards the controller is adapted to be applied to the regulation and tracking control problems. For stable systems, controllers could be designed to improve the system response, and afterwards the controller is adapted to be applied to the regulation and tracking control problems.

The general structure of feedback control system is shown in Figure 1. A neural controller, having as inputs the system output (or state) y , and the desired reference (or state) r , will produce a control input u to be applied to the system so that its output y follows the desired reference r as close as possible.



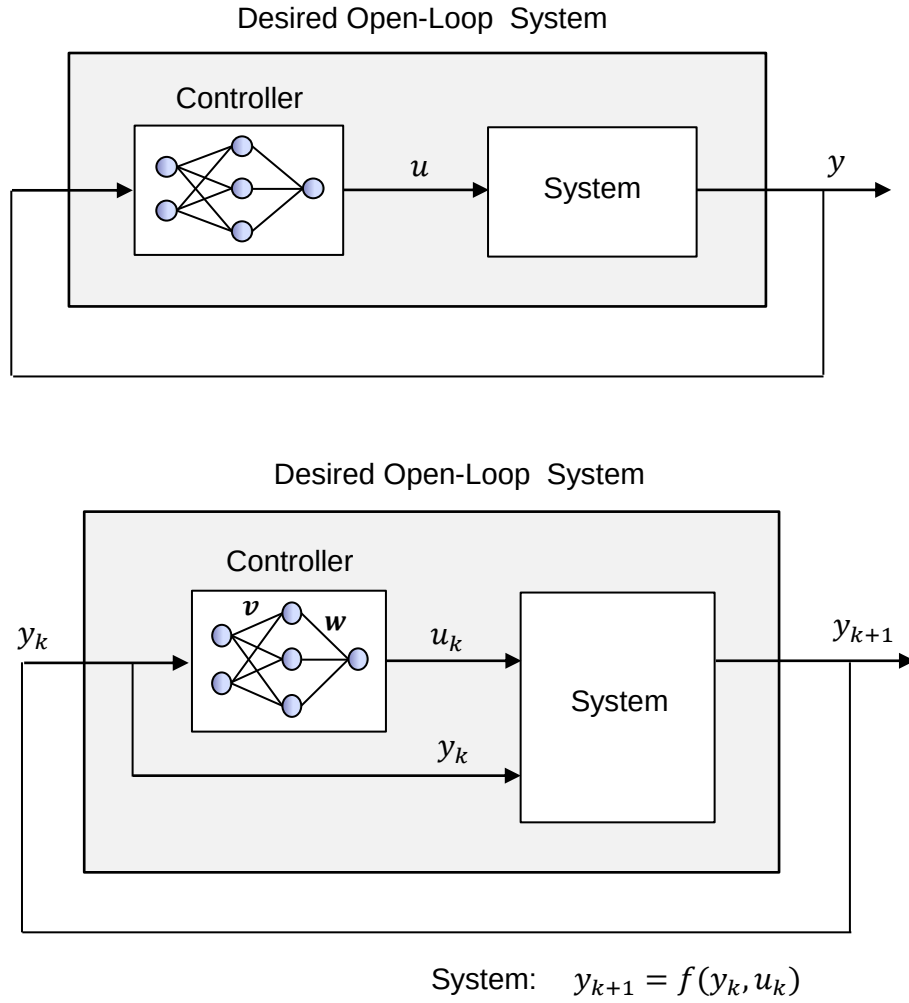
The neural controller could be a static or dynamic neural network. In this chapter, static neural controllers will be considered: the control signal u depends only on present external inputs y and r . Although the controller is a static network, the closed-loop system is a dynamic one with outputs becoming inputs in the next step.

The neural controller can be trained in a static learning or dynamic learning fashion. Static learning is a simple but approximate learning method that does not take into account the dynamics of the complete control problem. The dynamic learning method is mathematically more consistent, and takes into account the dynamics of the closed-loop system, as well as the dynamics of the system to be controlled. Both learning methods are model-based, i.e., the controller design requires the knowledge of the system model which may be represented by an equation or by a neural network. In general, the system model is represented by the following equation:

$$y_{k+1} = f(y_k, u_k) \quad (1)$$

In the following, each learning method will be presented.

Static Learning Method



The neural controller is trained so that the open-loop controller-system integration (shaded box) has the same input-output mapping as the input-output mapping of the open-loop desired system. The desired system could be given by the following linear system:

$$y_{k+1}^* = A_m y_k \quad (2)$$

where A_m is the state matrix of the desired open-loop system, and y_k^* is its output. The neural controller is trained to minimize the following cost function J :

$$J = \frac{1}{2} \sum_{k=0}^{N-1} (y_{k+1} - y_{k+1}^*)^T Q (y_{k+1} - y_{k+1}^*) + u_k^T R u_k \quad (3)$$

where Q and R are positive matrices selected based on the desired performance

The neural controller may also be trained so that the following cost function J is minimized for the open-loop controller-system integration (shade box):

$$J = \frac{1}{2} \sum_{k=0}^{N-1} y_{k+1}^T Q y_{k+1} + u_k^T R u_k \quad (4)$$

It is noted that equation (4) is obtained by making y_{k+1}^* equal to zero in equation (3).

In static training, the dynamics of the system to be controlled and of the closed-loop system are not considered. Therefore, the neural controller is trained using simple partial derivatives only. The weighting coefficients v and w are updated using the steepest descent method as follows:

$$v = v - \eta \frac{\partial J}{\partial v} \quad (5)$$

$$w = w - \eta \frac{\partial J}{\partial w} \quad (6)$$

where the simple partial $\partial J/\partial v$ and $\partial J/\partial w$ are computed at each step without considering any dynamics of the closed-loop control problem. The standard error back propagation algorithm may be applied in both pattern and batch learning methods as it was presented in Chapter 2. Pattern learning is an approximate training method where the cost function of equation (3) is defined for each time step k as follows.

$$J_k = \frac{1}{2} (y_{k+1} - y_{k+1}^*)^T Q (y_{k+1} - y_{k+1}^*) + \frac{1}{2} u_k^T R u_k \quad k = 0, 1, 2, \dots, N-1 \quad (7)$$

and the simple partial derivatives can be computed at each iteration step from the equation:

$$\partial J_k / \partial v = (y_{k+1} - y_{k+1}^*)^T Q \frac{\partial y_{k+1}}{\partial v} + u_k^T R \frac{\partial u_k}{\partial v} \quad (8)$$

$$\frac{\partial J_k}{\partial w} = (y_{k+1} - y_{k+1}^*)^T Q \frac{\partial y_{k+1}}{\partial w} + u_k^T R \frac{\partial u_k}{\partial w} \quad (9)$$

with

$$\frac{\partial y_{k+1}}{\partial v} = \frac{\partial y_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial v} \quad (10)$$

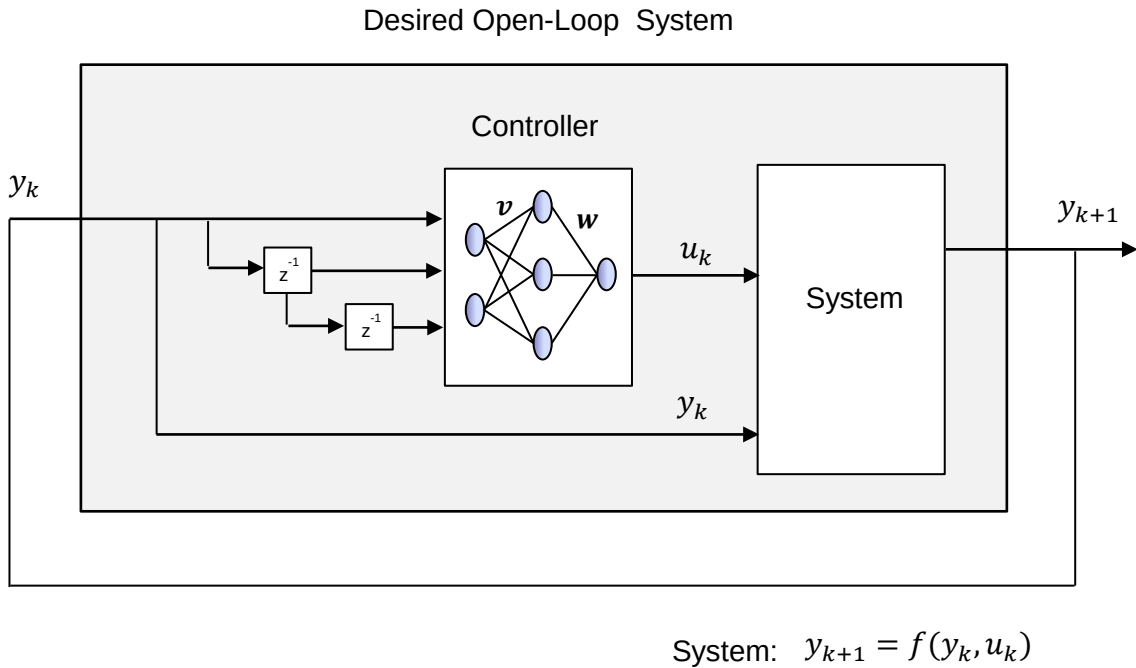
$$\frac{\partial y_{k+1}}{\partial w} = \frac{\partial y_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial w} \quad (11)$$

where derivative $\partial y_{k+1}/\partial u_k$ is computed from the system model of equation (1), and derivatives $\partial u_k/\partial v$ and $\partial u_k/\partial w$ are computed from the neural controller of which all its equations are known. As it was stated in Chapter 2, in pattern learning the weighting coefficients v and w are updated for each training patter, i.e., for each value of k .

For batch learning, the simple partial derivatives of equations (8) and (9) are accumulated for all training set ($k = 0, 1, 2, \dots, N-1$) to proceed afterwards with the updating of weighting coefficients v and w .

The static neural controller can have as inputs not only system present output (or state) y_k but also system past outputs as it is shown in Figure where it is noted that the controller has as inputs y_k ,

y_{k-1} , and y_{k-2} . This controller is trained applying the same static training methods presented above.



Although the neural controller static learning is simple to implement, it does not consider the actual dynamics of the closed-loop system and therefore it presents some drawbacks that should be taken into account. The most important issues are:

- (a) The controller training should be monitored given that the cost function is not always decreasing along the training process, and training should be stopped.
- (b) Static learning is mostly applied to stable systems for improving their response or for regulation and tracking control problems.
- (c) Usually, static learning does not perform well for unstable systems, given that, in this case, the actual dynamics of the system to be controlled and of the closed-loop system must be taken into account. Dynamic learning, to be presented in next section, performs well for unstable systems.

Neural Controller Static Training

Initial positions for training

Train the neural controller for a set of initial positions. Afterwards, the neural controller should be able to generalize to other initial positions.

In the following programs, four initial positions have been considered.

Incremental learning

Similarly as human beings, neural networks must be trained starting with simple tasks to continue with more complex task in an incremental fashion.

For most of systems, simple tasks are those close to a stable state.

Problema:

Controlar sistema bilineal:

$$x_{k+1} = Ax_k + Bu_k + Gx_k u_k$$

Sistema estable con no muy Buena respuesta transitoria.

Se requiere entrenar una red neuronal para controlar el sistema

- Mejorar su respuesta transitoria
DynamicBPControlEstatico0.m

- Tracking control $x \rightarrow x^*$ $u \rightarrow u^*$

Los valores de x^* y u^* deben cumplir la condición:

$$x^* = Ax^* + Bu^* + Gx^* u^*$$

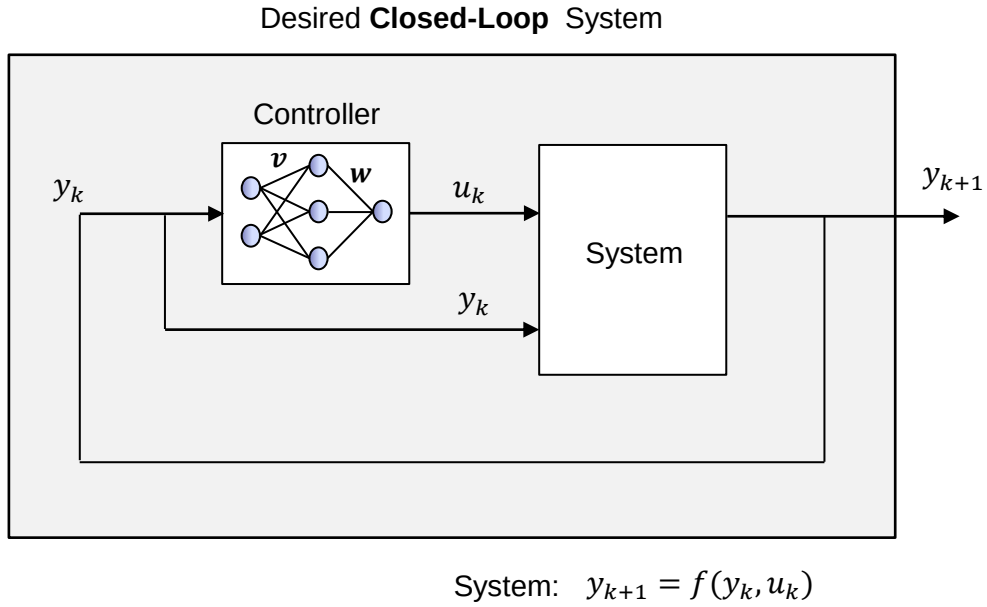
DynamicBPControlEstatico1.m

- Entrenar también controlador con entradas pasadas
DynamicBPControlEstatico3.m

Dynamic Learning Method

In dynamic training the neural controller is trained so that the **closed-loop** system has the desired response. The dynamic training method takes into account the dynamics of the closed-loop system, as well as the dynamics of the system to be controlled. It is mathematically consistent and it uses the total partial derivatives for updating the neural network weighting coefficients.

Figure shows the general structure of dynamic learning method which takes into account the inherent dynamics of the control problem: the output y_k of the system becomes input in the next step for both the same systems and the controller.



The neural controller is trained so that the closed-loop controller-system integration (shaded box) has the same input-output mapping as the input-output mapping of the closed-loop desired system. The desired system is selected to have the desired response physically achievable by the systems to be controlled. It could be represented by a linear equation as follows:

$$y_{k+1}^* = A_m y_k \quad (12)$$

where A_m is the state matrix of the desired closed-loop system, and y_k^* is its output.

The neural controller is trained to minimize the following cost function J :

$$J = \frac{1}{2} (y_N - y_N^*)^T Q (y_N - y_N^*) + \frac{1}{2} \sum_{k=0}^{N-1} (y_k - y_k^*)^T Q (y_k - y_k^*) + u_k^T R u_k \quad (13)$$

where Q and R are positive matrices selected based on the desired performance.

The neural controller may also be trained so that the following cost function J is minimized for the closed-loop system:

$$J = \frac{1}{2} \sum_{k=0}^{N-1} y_{k+1}^T Q y_{k+1} + u_k^T R u_k \quad (14)$$

It is noted that equation (4) is obtained by making y_{k+1}^* equal to zero in equation (13).

Since in dynamic training, the dynamics of the system to be controlled and of the closed-loop system are considered, the neural controller is trained using the total partial derivatives. Therefore, the weighting coefficients v and w are updated using the steepest descent method as follows:

$$\begin{aligned} v &= v - \eta \frac{\partial J}{\partial v} \\ w &= w - \eta \frac{\partial J}{\partial w} \end{aligned} \tag{15}$$

where $\frac{\partial J}{\partial v}$ and $\frac{\partial J}{\partial w}$ represent the total partial derivatives of J respect to weighting coefficients v and w , respectively. Differentiating J in equation (13), the following total partial derivatives are obtained:

$$\frac{\partial J}{\partial w} = \sum_{k=1}^N (y_k - y_k^*)^T Q \frac{\partial y_k}{\partial w} + \sum_{k=0}^{N-1} u_k^T R \frac{\partial u_k}{\partial w} \tag{16}$$

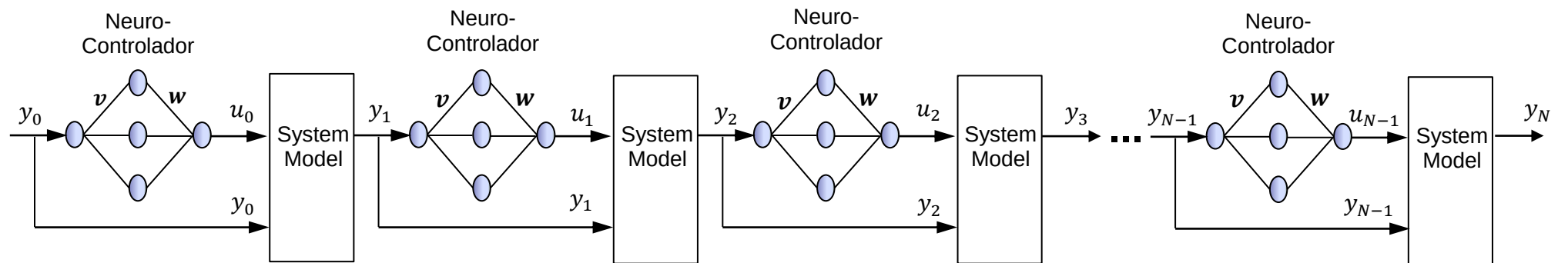
$$\frac{\partial J}{\partial v} = \sum_{k=1}^N (y_k - y_k^*)^T Q \frac{\partial y_k}{\partial v} + \sum_{k=0}^{N-1} u_k^T R \frac{\partial u_k}{\partial v} \tag{17}$$

The total partial derivatives $\frac{\partial y_k}{\partial w}$, $\frac{\partial y_k}{\partial v}$, $\frac{\partial u_k}{\partial w}$ and $\frac{\partial u_k}{\partial v}$ are computing considering the dynamics of the closed-loop control problem.

Time Deployment of Dynamic Neural Networks

For formulating the training algorithm of dynamic neural networks it is important to understand how they evolve along time.

Figure shows the time deployment of the closed-loop system, that integrates the neural controller and the dynamic system, starting from initial state y_0 . It is noted that the output of the closed-loop system becomes input at the next step.



Step 1

Step 2

Step 3

...

Step N

Network 1

Network 1-2

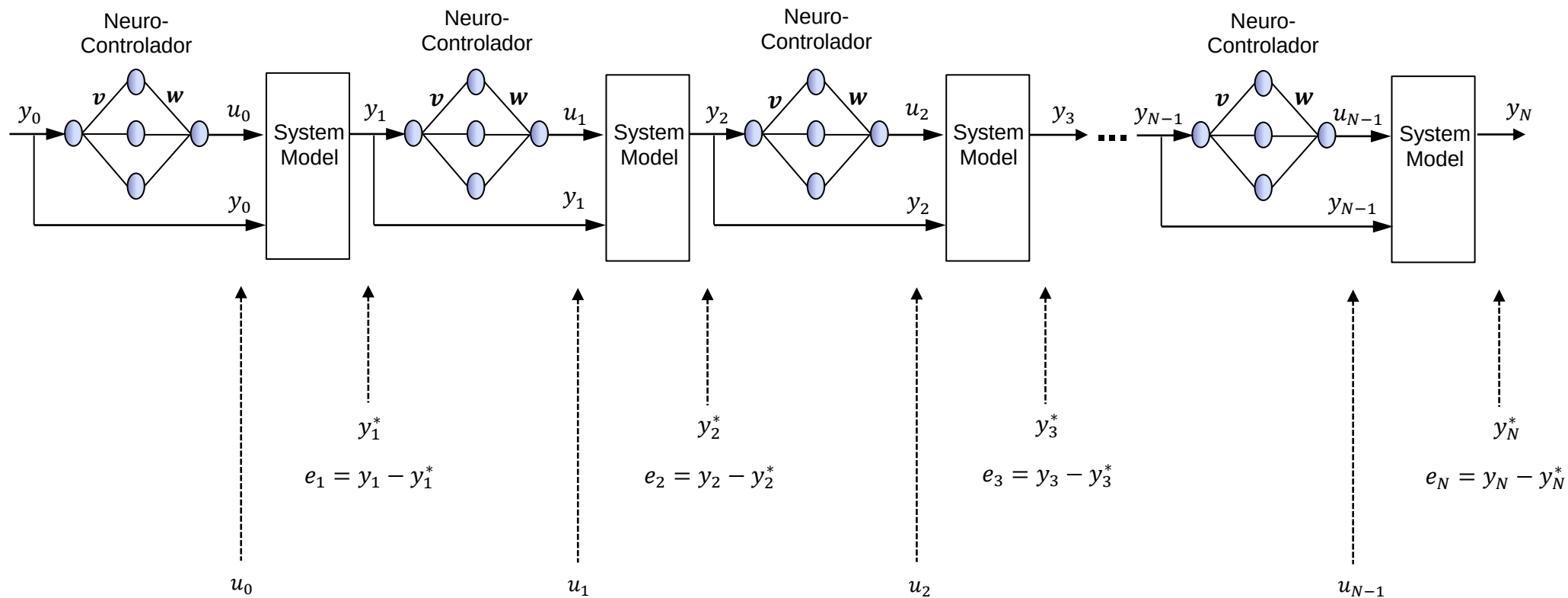
Network 1-2-3

In Figure 2 it is noted that:

- The same closed-loop system is unfolded at each time step: same neural controller and same dynamic system. Neural controller weighting coefficients v and w are the same for each time step.
- The output at step k becomes input at step $k + 1$.
- If the networks of steps 1 and 2 are considered as a single network, this enlarged network has as inputs y_0 and output y_2 . Likewise, output y_2 depends on weighting coefficient w of the second-step network and on weighting coefficient w of the first-step network. However, the dependence is different for each of them: $\partial y_2 / \partial w$ for the weighting coefficient w of the second-step network is different to the $\partial y_2 / \partial w$ for the weighting coefficient w of the first-step network. The total partial derivative of y_2 respect to weighting coefficient w comprises the total dependence for all involved steps. The same is for the total partial derivative respect to weighting coefficient v .
- Similarly, if the networks of steps 1, 2 and 3 are considered as a single network, this enlarged network has as inputs y_0 and output y_3 . Likewise, output y_3 depends on weighting coefficient w of the third-step network, on weighting coefficient w of the second-step network, and on weighting coefficient w of the first-step network. However, the dependence is different for each of them: derivative $\partial y_3 / \partial w$ for the weighting coefficient w of the third-step network is different to derivative $\partial y_3 / \partial w$ for the weighting coefficient w of the second-step network, and they are also different to derivative $\partial y_3 / \partial w$ for the weighting coefficient w of the first-step network. In other words, even the value of w is the same for the three networks, the derivatives are different. It is owed to the time-dependent behavior of the dynamic neural-network. The total partial derivative of y_3 respect to weighting coefficient w comprises the total dependence for all involved steps. The same is for the total partial derivative respect to weighting coefficient v .

Similarly as with the training of recurrent neural networks, the Back Propagation Through Time BPTT algorithm and the Dynamic Back Propagation DBP algorithm can be applied for the dynamic training of neural controllers.

Back Propagation Through Time BPTT



Network 1

Network 1-2

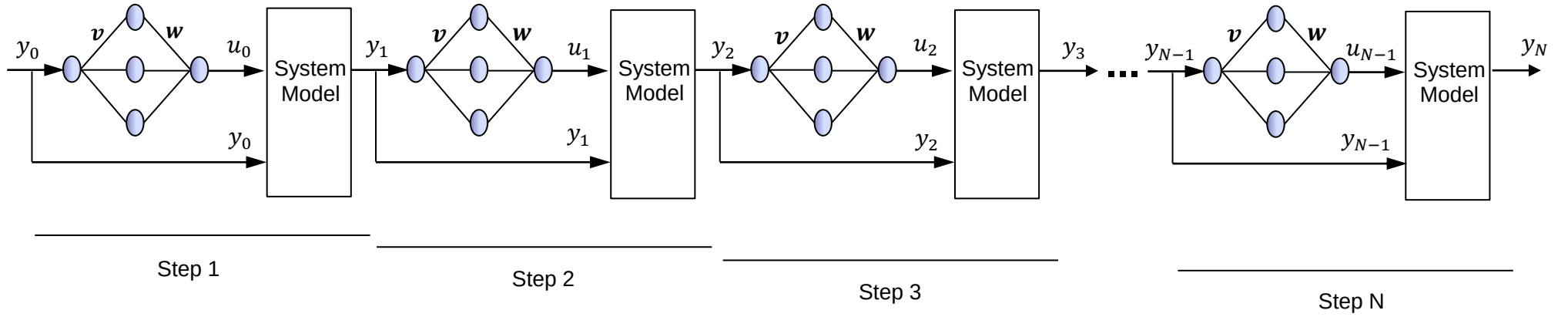
Network 1-2-3

Dynamic Back Propagation DBP

In the Dynamic Back Propagation algorithm the total partial derivatives of $\overline{\partial y_k / \partial w}$, $\overline{\partial y_k / \partial v}$, $\overline{\partial u_k / \partial w}$ and $\overline{\partial u_k / \partial v}$ required for updating the weighting coefficients v and w in equations (15) to (17) are computed in a recursive way using the following recursive expression:

$$\frac{\overline{\partial y_{k+1}}}{\partial w} = \frac{\partial y_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial w} + \left(\frac{\partial y_{k+1}}{\partial y_k} + \frac{\partial y_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial y_k} \right) \frac{\overline{\partial y_k}}{\partial w}$$

$$\frac{\overline{\partial u_k}}{\partial w} = \frac{\partial u_k}{\partial w} + \frac{\partial u_k}{\partial y_k} \frac{\overline{\partial y_k}}{\partial w}$$



$$\frac{\overline{\partial y_1}}{\partial w} = \frac{\partial y_1}{\partial u_0} \frac{\partial u_0}{\partial w}$$

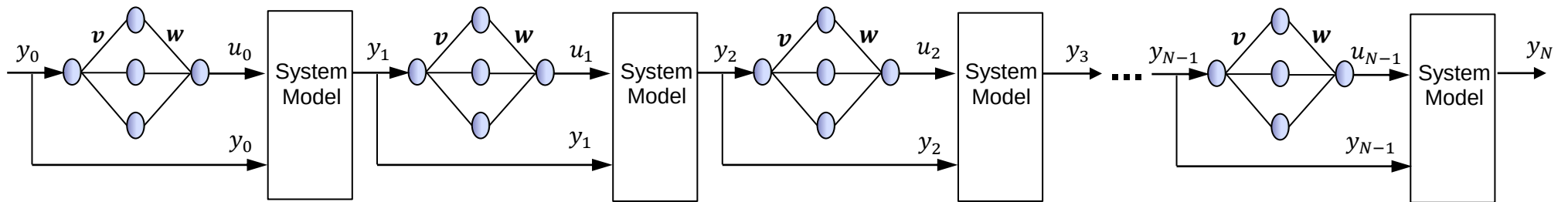
$$\frac{\overline{\partial y_2}}{\partial w} = \frac{\partial y_2}{\partial u_1} \frac{\partial u_1}{\partial w} + \frac{\partial y_2}{\partial y_1} \frac{\partial y_1}{\partial u_0} \frac{\partial u_0}{\partial w} + \frac{\partial y_2}{\partial u_1} \frac{\partial u_1}{\partial y_1} \frac{\partial y_1}{\partial u_0} \frac{\partial u_0}{\partial w}$$

$$\frac{\overline{\partial y_2}}{\partial w} = \frac{\partial y_2}{\partial u_1} \frac{\partial u_1}{\partial w} + \left(\frac{\partial y_2}{\partial y_1} + \frac{\partial y_2}{\partial u_1} \frac{\partial u_1}{\partial y_1} \right) \frac{\partial y_1}{\partial u_0} \frac{\partial u_0}{\partial w}$$

$$\frac{\overline{\partial y_2}}{\partial w} = \frac{\partial y_2}{\partial u_1} \frac{\partial u_1}{\partial w} + \left(\frac{\partial y_2}{\partial y_1} + \frac{\partial y_2}{\partial u_1} \frac{\partial u_1}{\partial y_1} \right) \frac{\overline{\partial y_1}}{\partial w}$$

$$\frac{\overline{\partial y_3}}{\partial w} = \frac{\partial y_3}{\partial u_2} \frac{\partial u_2}{\partial w} + \left(\frac{\partial y_3}{\partial y_2} + \frac{\partial y_3}{\partial u_2} \frac{\partial u_2}{\partial y_2} \right) \frac{\overline{\partial y_2}}{\partial w}$$

$$\frac{\overline{\partial y_{k+1}}}{\partial w} = \frac{\partial y_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial w} + \left(\frac{\partial y_{k+1}}{\partial y_k} + \frac{\partial y_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial y_k} \right) \frac{\overline{\partial y_k}}{\partial w}$$



$$\overline{\frac{\partial u_0}{\partial w}} = \frac{\partial u_0}{\partial w}$$

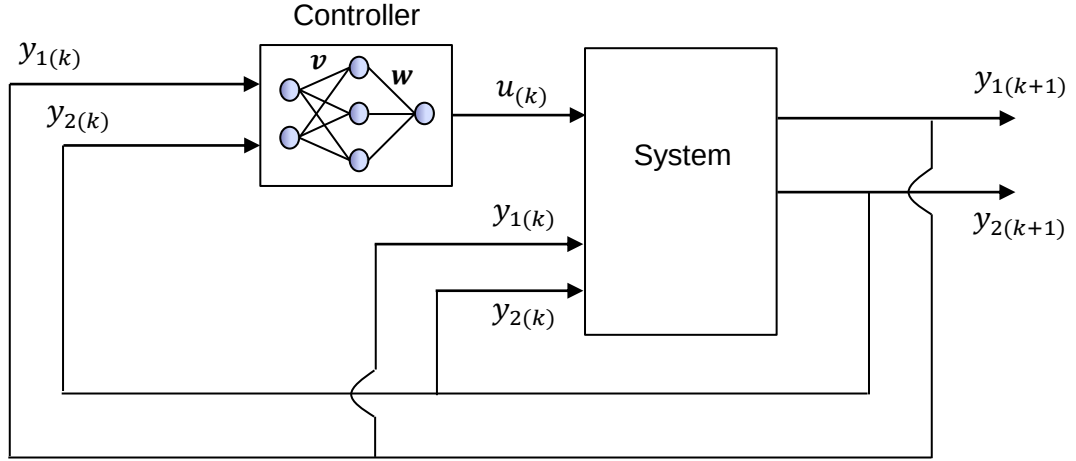
$$\overline{\frac{\partial u_1}{\partial w}} = \frac{\partial u_1}{\partial w} + \frac{\partial u_1}{\partial y_1} \frac{\partial y_1}{\partial u_0} \frac{\partial u_0}{\partial w}$$

$$\overline{\frac{\partial u_1}{\partial w}} = \frac{\partial u_1}{\partial w} + \frac{\partial u_1}{\partial y_1} \overline{\frac{\partial y_1}{\partial w}}$$

$$\overline{\frac{\partial u_2}{\partial w}} = \frac{\partial u_2}{\partial w} + \frac{\partial u_2}{\partial y_2} \left[\frac{\partial y_2}{\partial u_1} \frac{\partial u_1}{\partial w} + \left(\frac{\partial y_2}{\partial y_1} + \frac{\partial y_2}{\partial u_1} \frac{\partial u_1}{\partial y_1} \right) \frac{\partial y_1}{\partial u_0} \frac{\partial u_0}{\partial w} \right]$$

$$\overline{\frac{\partial u_2}{\partial w}} = \frac{\partial u_2}{\partial w} + \frac{\partial u_2}{\partial y_2} \overline{\frac{\partial y_2}{\partial w}}$$

$$\overline{\frac{\partial u_k}{\partial w}} = \frac{\partial u_k}{\partial w} + \frac{\partial u_k}{\partial y_k} \overline{\frac{\partial y_k}{\partial w}}$$



$$\text{System: } y_{(k+1)} = f(y_{(k)}, x_{(k)}) \quad y_{(k)} = \begin{bmatrix} y_{1(k)} \\ y_{2(k)} \end{bmatrix}$$

The recursive equation for computing total partial derivatives:

$$\frac{\overline{\partial y_{(k+1)}}}{\partial w} = \frac{\partial y_{(k+1)}}{\partial u_{(k)}} \frac{\partial u_{(k)}}{\partial w} + \left(\frac{\partial y_{(k+1)}}{\partial y_{(k)}} + \frac{\partial y_{(k+1)}}{\partial u_{(k)}} \frac{\partial u_{(k)}}{\partial y_{(k)}} \right) \frac{\overline{\partial y_{(k)}}}{\partial w}$$

$$\frac{\overline{\partial y_{(k)}}}{\partial w} = \begin{bmatrix} \frac{\overline{\partial y_{1(k)}}}{\partial w} \\ \frac{\overline{\partial y_{2(k)}}}{\partial w} \end{bmatrix}$$

Jacobian matrix:

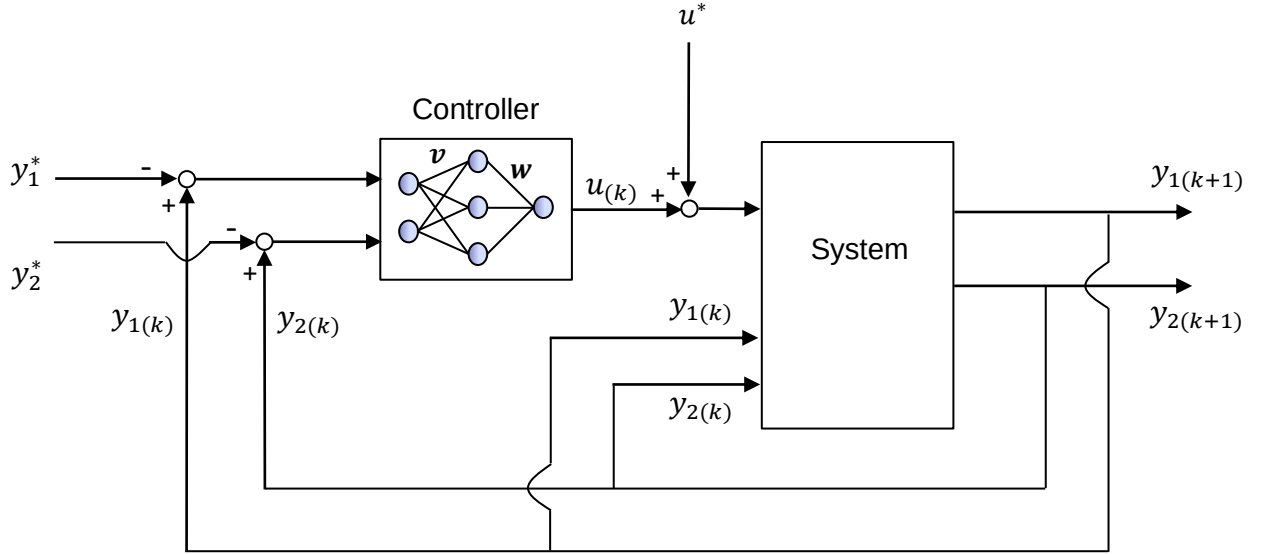
$$\frac{\partial y_{(k+1)}}{\partial y_{(k)}} = \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{1(k+1)}}{\partial y_{2(k)}} \\ \frac{\partial y_{2(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{2(k+1)}}{\partial y_{2(k)}} \end{bmatrix}$$

$$\begin{bmatrix} \frac{\overline{\partial y_{1(k+1)}}}{\partial w} \\ \frac{\overline{\partial y_{2(k+1)}}}{\partial w} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial u_{(k)}} \\ \frac{\partial y_{2(k+1)}}{\partial u_{(k)}} \end{bmatrix} \frac{\partial u_{(k)}}{\partial w} + \left(\begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{1(k+1)}}{\partial y_{2(k)}} \\ \frac{\partial y_{2(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{2(k+1)}}{\partial y_{2(k)}} \end{bmatrix} + \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial u_{(k)}} \\ \frac{\partial y_{2(k+1)}}{\partial u_{(k)}} \end{bmatrix} \begin{bmatrix} \frac{\partial u_{(k)}}{\partial y_{1(k)}} & \frac{\partial u_{(k)}}{\partial y_{2(k)}} \end{bmatrix} \right) \begin{bmatrix} \frac{\overline{\partial y_{1(k)}}}{\partial w} \\ \frac{\overline{\partial y_{2(k)}}}{\partial w} \end{bmatrix}$$

$$\begin{bmatrix} \frac{\overline{\partial y_{1(k+1)}}}{\partial v} \\ \frac{\overline{\partial y_{2(k+1)}}}{\partial v} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial u_{(k)}} \\ \frac{\partial y_{2(k+1)}}{\partial u_{(k)}} \end{bmatrix} \frac{\partial u_{(k)}}{\partial v} + \left(\begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{1(k+1)}}{\partial y_{2(k)}} \\ \frac{\partial y_{2(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{2(k+1)}}{\partial y_{2(k)}} \end{bmatrix} + \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial u_{(k)}} \\ \frac{\partial y_{2(k+1)}}{\partial u_{(k)}} \end{bmatrix} \begin{bmatrix} \frac{\partial u_{(k)}}{\partial y_{1(k)}} & \frac{\partial u_{(k)}}{\partial y_{2(k)}} \end{bmatrix} \right) \begin{bmatrix} \frac{\overline{\partial y_{1(k)}}}{\partial v} \\ \frac{\overline{\partial y_{2(k)}}}{\partial v} \end{bmatrix}$$

Tracking Control

The neural controller trained for stabilization or regulation can be used for tracking control for properly changing the neural controller inputs and output.



$$\text{System: } y_{(k+1)} = f(y_{(k)}, u_{(k)}) \quad y_{(k)} = \begin{bmatrix} y_{1(k)} \\ y_{2(k)} \end{bmatrix}$$

If the trained neural controller is represented by:

$$u_{(k)} = h(y_k)$$

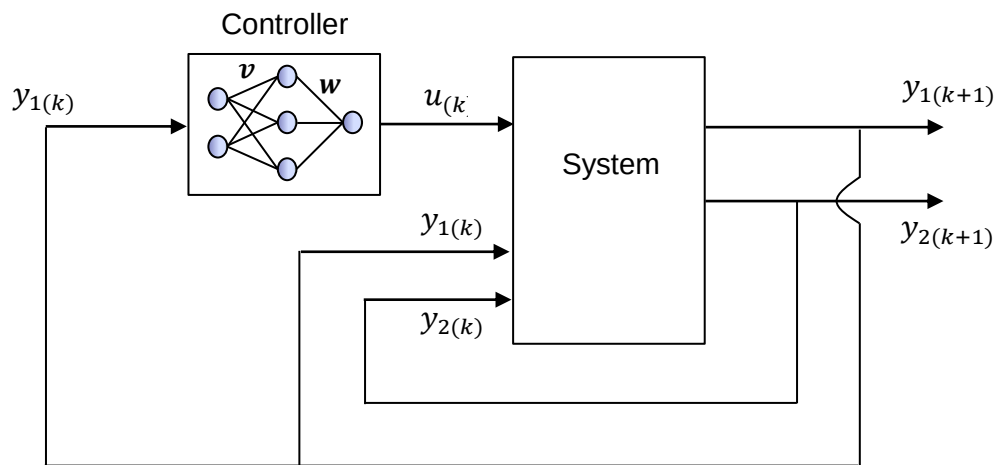
the control law to be applied in tracking control is:

$$u_{(k)} - u^* = h(y_k - y^*)$$

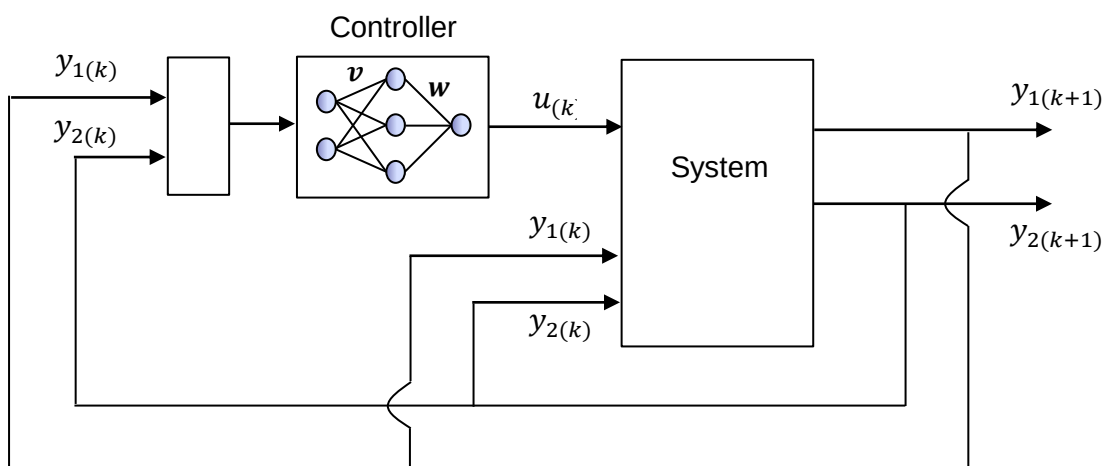
where the desired values y^* and u^* satisfy the following condition:

$$y^* = f(y^*, u^*)$$

Partial Output Feedback Control



$$\text{System: } y_{(k+1)} = f(y_{(k)}, x_{(k)}) \quad y_{(k)} = \begin{bmatrix} y_1(k) \\ y_2(k) \end{bmatrix}$$



$$\text{System: } y_{(k+1)} = f(y_{(k)}, x_{(k)}) \quad y_{(k)} = \begin{bmatrix} y_1(k) \\ y_2(k) \end{bmatrix}$$

Neural Controller Dynamic Training

Initial positions for training

Train the neural controller for a set of initial positions. Afterwards, the neural controller should be able to generalize to other initial positions.

In the following programs, four initial positions have been considered.

Incremental learning

Similarly as human beings, neural networks must be trained starting with simple tasks to continue with more complex task in an incremental fashion.

For most of systems, simple tasks are those close to a stable state.

Problema:

Controlar sistema bilineal inestable:

$$x_{k+1} = Ax_k + Bu_k + Gx_k u_k$$

Se requiere entrenar una red neuronal para estabilizar el sistema.

- Entrenamiento de neuro-controlador con full-state feedback.
DynamicBPControl2.m

- Tracking control $x \rightarrow x^*$ $u \rightarrow u^*$

Los valores de x^* y u^* deben cumplir la condición:

$$x^* = Ax^* + Bu^* + Gx^* u^*$$

DynamicBPControl2.m

- Entrenamiento de neuro-controlador con salida parcial estabilizar un sistema inestable.

DynamicBPControl2a.m

- Validación del neuro-controlador para perturbaciones externas.

DynamicBPControl2b.m

Problema:

Control de robot móvil:

$$\dot{x} = v \cos \varphi$$

$$\dot{y} = v \sin \varphi$$

$$\dot{\varphi} = -\frac{v}{L} \tan \delta$$

$$x_{k+1} = x_k + \frac{r}{L} \cos(\varphi_k)$$

$$y_{k+1} = y_k + \frac{r}{L} \sin(\varphi_k)$$

$$\varphi_{k+1} = \varphi_k - \frac{r}{L} \tan \delta_k$$

Programa de Entrenamiento: DynamicBPcarro.m

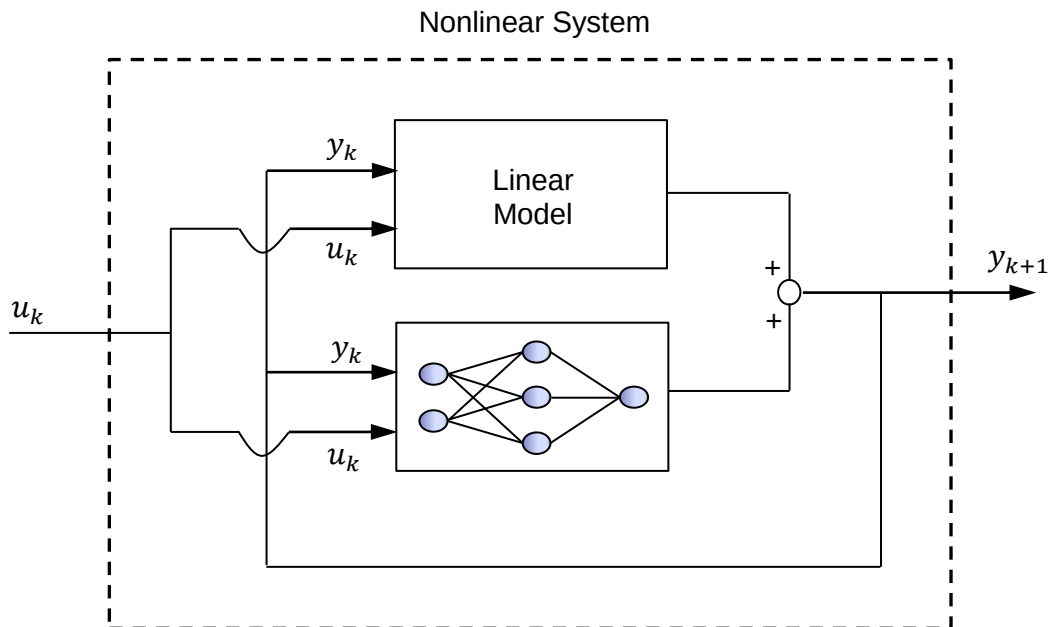
Programa de Validación: DynamicBPCarroValida.m

Integration of Linear Systems and Neural Networks for the Identification and Control of Nonlinear Systems

Nonlinear Modeling

Nonlinear systems can be approximated by a linear system around a set point. This linear model is a good approximation of the system around this point only. However, to expand the accuracy of the model for a wide range of operating conditions, the linear system can be integrated in parallel with a neural network so that the linear + neural network combination represents a better model of the system.

Figure shows the parallel integration of a linear system and a neural network for representing the dynamics of a nonlinear system. The linear system is determined by modeling the system under certain linear assumptions and it depends mostly on system parameters. The neural network, however, is nonlinear and it is trained using input-output data of the actual nonlinear system, and considering the equation of the linear system.



The equation representing the integrated model of the nonlinear system is represented by:

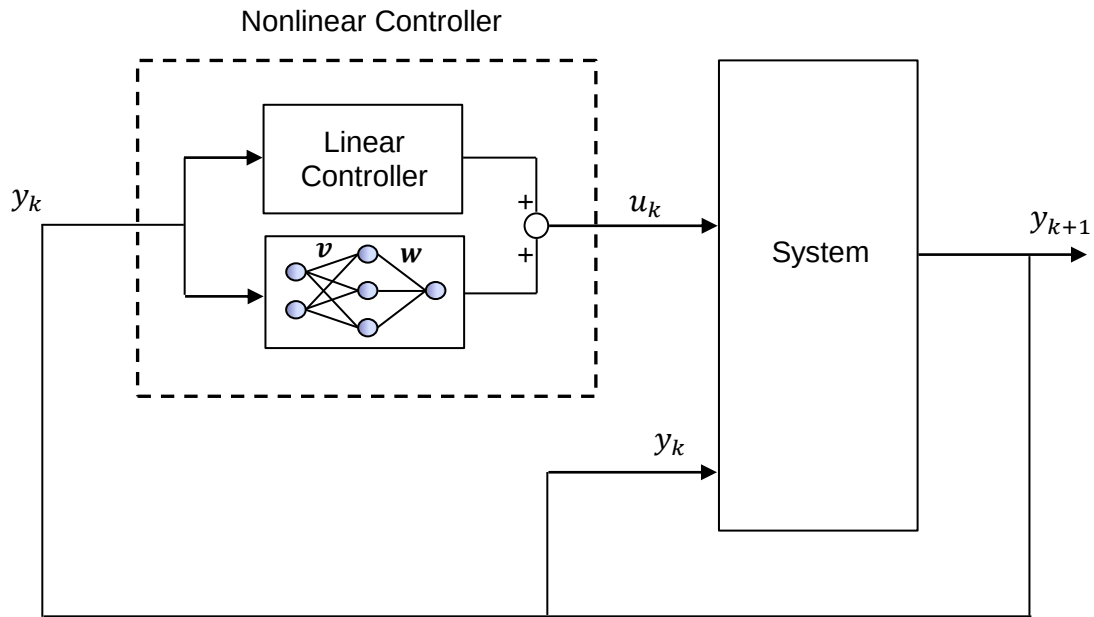
$$y_{k+1} = Ay_k + Bu_k + g(y_k, u_k)$$

where the nonlinear component $g(y_k, u_k)$ is represented by the neural network. The integrated model is a better representation of the nonlinear linear system, and it is based on knowledge represented by the linear model, and on data used for training the neural network.

Nonlinear Control

Linear controllers can also be integrated in parallel with neural networks for designing nonlinear controllers for nonlinear systems. The linear component could be determined using a linearized model and applying any of the different methods for designing linear controllers for linear systems. This linear controller will perform well around the state the linearized system was formulated. However, to expand and improve the performance of the linear controller it is integrated in parallel with a nonlinear neural network.

Figure shows the parallel integration of a linear controller and a neural network for designing a nonlinear controller for a nonlinear system. The neural network can be trained so that the closed-loop systems optimizes a given performance measure.



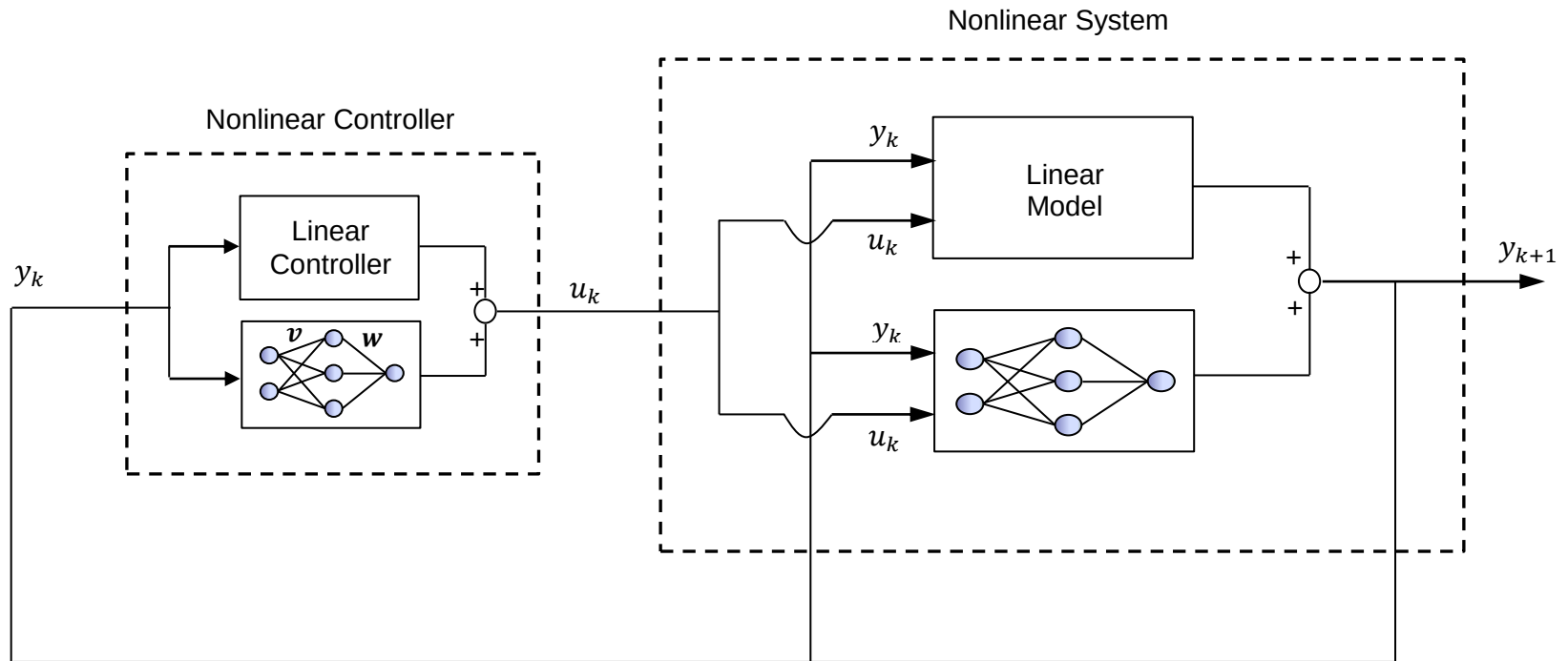
$$\text{System: } y_{k+1} = f(y_k, u_k)$$

The equation representing the integrated linear and neural network controller is given by:

$$u_k = -Ky_k + h(y_k)$$

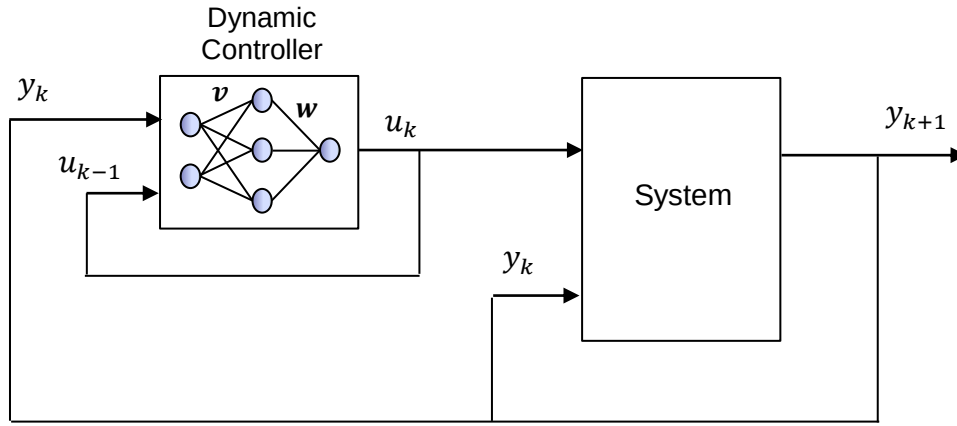
where the nonlinear component $h(y_k)$ is represented by the neural network trained using the nonlinear model of the system, and considering the linear controller. It is expected the integrated controller to have a better performance than the linear controller when applied to the nonlinear system.

Integration of Linear + Neural Network Nonlinear Model and Linear + Neural Network Nonlinear Controller



Dynamic Neural Controller

Dynamic neural networks can also be trained for controlling nonlinear dynamic system as it is shown in Figure where it is noted that the output u_k of the controller becomes its input at the next time step.



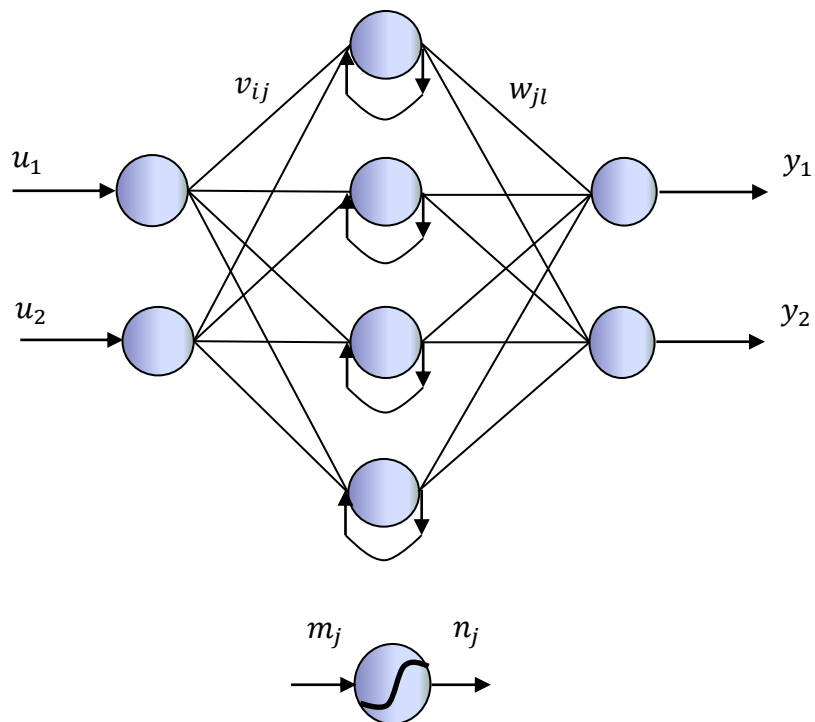
$$\text{System: } y_{k+1} = f(y_k, u_k)$$

The equation of the dynamic neural controller is given by:

$$u_k = h(u_{k-1}, y_k)$$

and it is trained considering the dynamics of the system for optimizing a given performance measure.

Other Representations of Recurrent Systems



$$n_{j(k)} = f(m_{j(k)})$$

$$m_{j(k)} = n_{j(k-1)} + \sum_{i=1}^2 v_{ij} u_i$$

LSTM and GRU

LSTM: Long Short-Term Memory

GRU: Gated Recurrent Unit

<https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21>